Advanced Formal Tools

CTL Model Checker

Jing Wen
jing.wen@etu.unige.ch

Nikolay.Chikin
nikolay.chikin@etu.unige.ch

University of Geneva

CONTENTS

01

PROJECT REVIEW

02

CTL FORMULA READER

03

FSM MODEL READER

04

FORMULA EVALUATION

05

TESTING

06

SCELDULE AND CONCLUSION

Project review

CTL model checking algorithm

- **Definition:**

Based mainly on forward state traversal;
Can check many realistic CTL properties without doing backward state traversal

- **The value:**

Fewer resources, faster speed;
Verify actual CTL properties of large industrial models

Project review

CTL model checking algorithm--based on forward state traversal

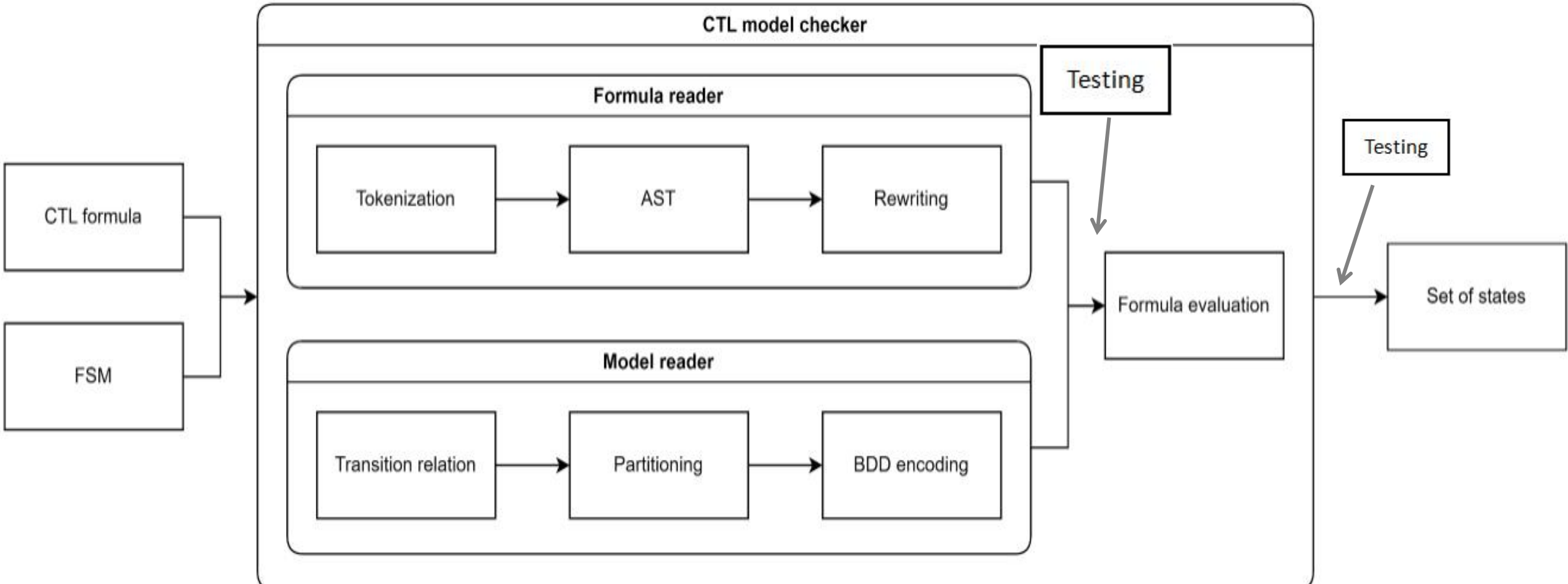
- **Demonstrate:**

Environment: Gitlab

1. README.md
link: https://gitlab.unige.ch/Nikolay.Chikin/ctl-model-checker/-/blob/main/README.md
2. .gitlab-ci.yml
link: https://gitlab.unige.ch/Nikolay.Chikin/ctl-model-checker/-/blob/main/.gitlab-ci.yml

Project review

CTL model checking algorithm--based on forward state traversal



PART1 -- CTL formula reader

- **Definition:** Reading and parsing CTL formulas

- **Key Techniques:**

1. Syntax analysis (AST construction)

2. Rewriting: Simplify complex CTL formulas

1.double negations:

$$\neg(\neg f) = f$$

2.De Morgan's laws:

$$f \wedge g = \neg(\neg f \vee \neg g)$$
$$f \vee g = \neg(\neg f \wedge \neg g)$$

3.other logic equations:

$$EFf = E[trueUf]$$
$$AXf = \neg EX\neg f$$
$$AFf = \neg EG\neg f$$
$$AGf = \neg EF\neg f$$
$$A[gUf] = \neg(E[\neg fU\neg g \wedge \neg f] \vee EG\neg f)$$
$$x \rightarrow y = \neg x \vee y$$
$$x \leftrightarrow y = (x \wedge y) \vee (\neg x \wedge \neg y)$$

4. Replace path quantifiers:

$$s_0 \models f \iff p_0 \wedge f \neq \text{false}$$
$$s_0 \models f \iff p_0 \wedge \neg f = \text{false}$$

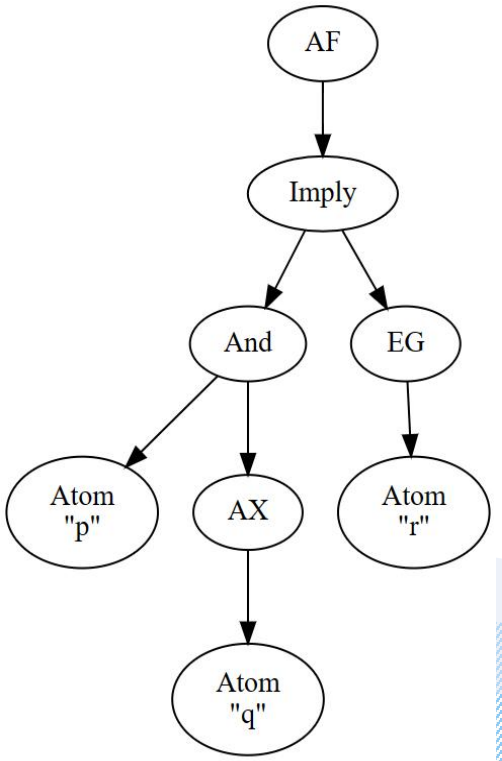
$$p \wedge \mathbf{E}[q\,\mathbf{U}\,f] \neq \text{false} \iff FwdUntil(p, q) \wedge f \neq \text{false}$$

$$p \wedge \mathbf{EX}\,f \neq \text{false} \iff Img(p) \wedge f \neq \text{false}$$

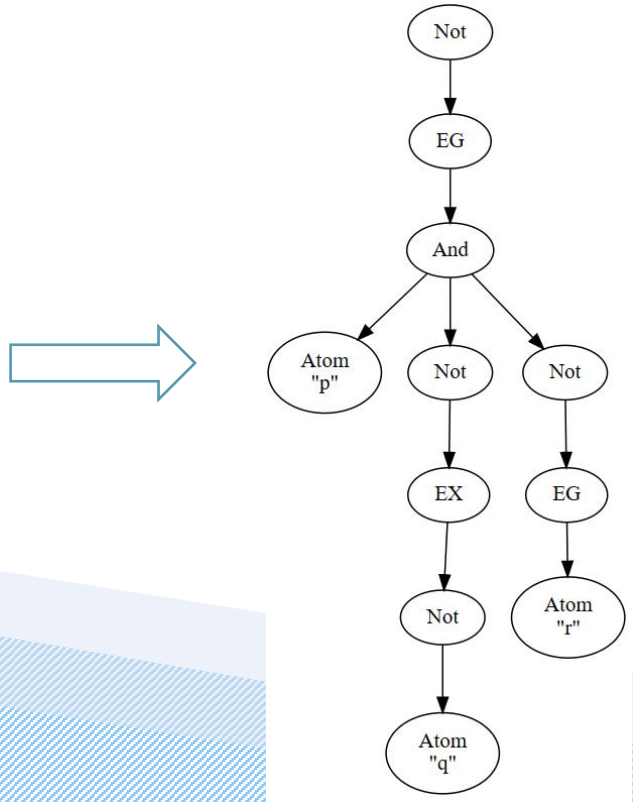
$$p \wedge \mathbf{EG}\,q \neq \text{false} \iff FwdGlobal(p, q) \neq \text{false}$$

PART1 -- CTL formula reader

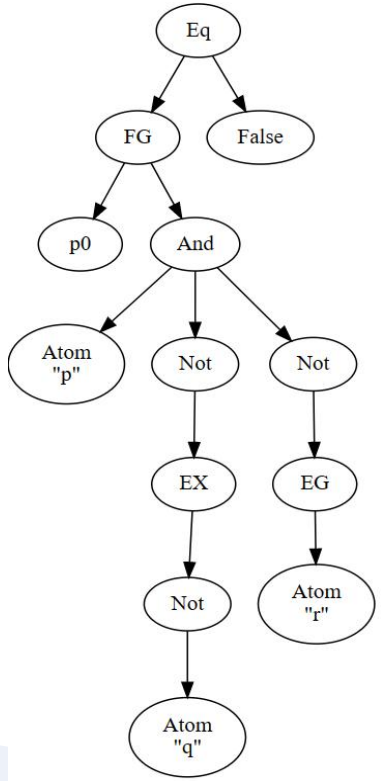
- **Example:** AF(p ∧ AXq ⇒ EGr)



original formula without rewriting



after rewriting logical operators and temporal operators



the final formula with using forward state traversal operator FG

PART2 -- FSM model reader

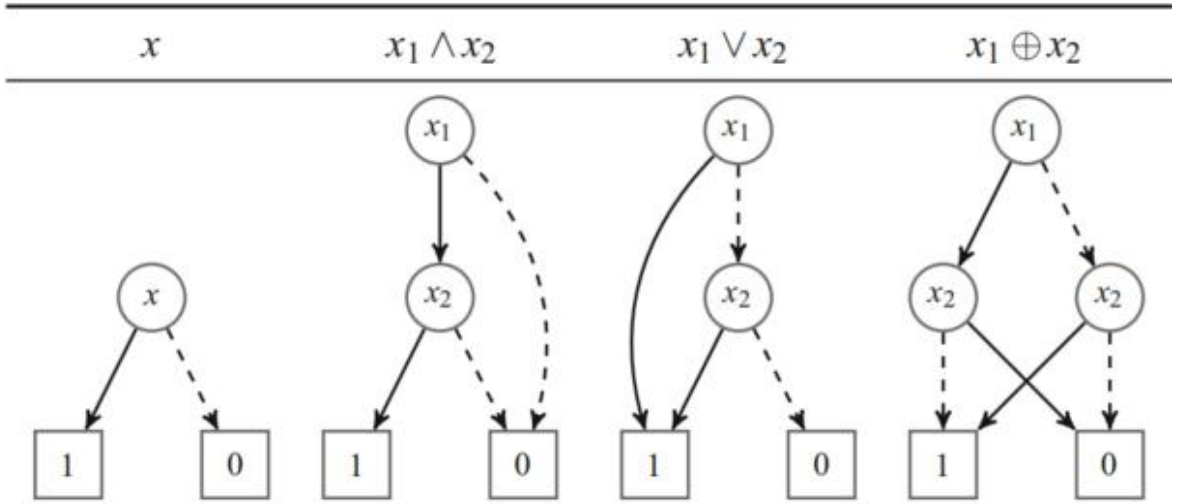
- **Definition:** Reading and processing the Finite State Machine (FSM) model

- **Key Techniques:**

1. State Encoding

2. Transition Relation Extraction

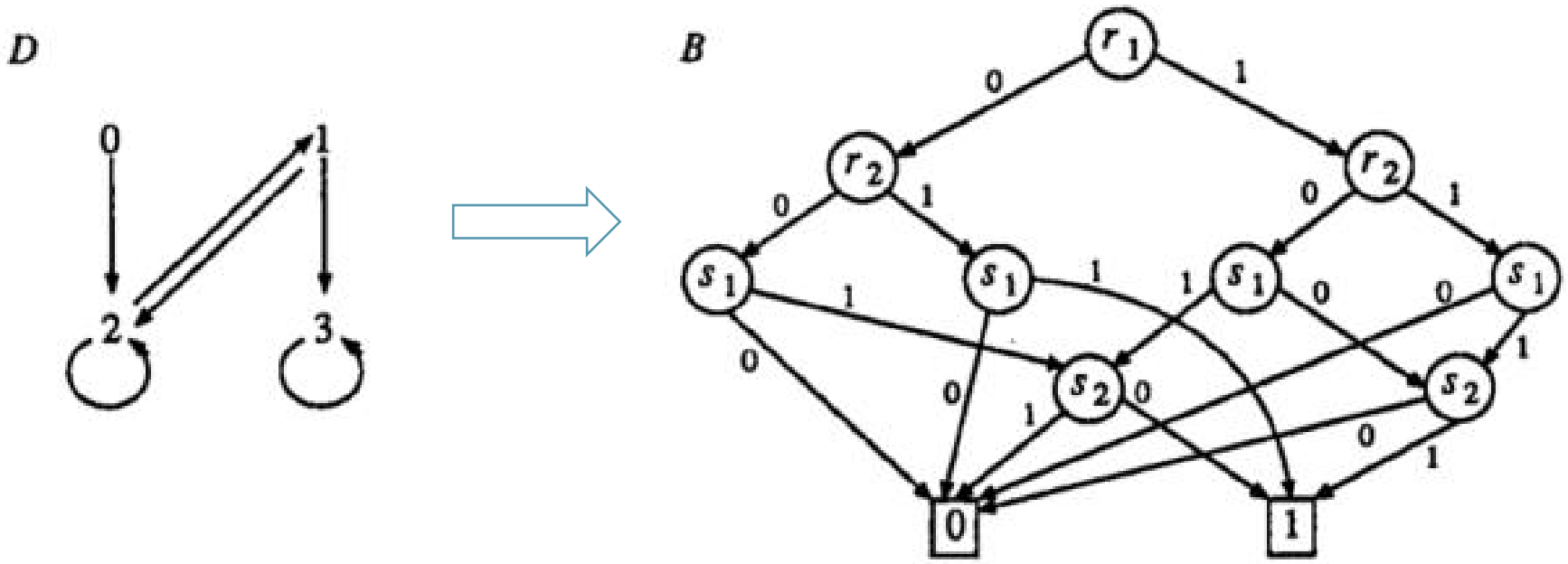
3. Symbolic Model Checking
(Binary Decision Diagrams (BDDs))



Binary decision diagrams for several Boolean functions

PART2 -- FSM model reader

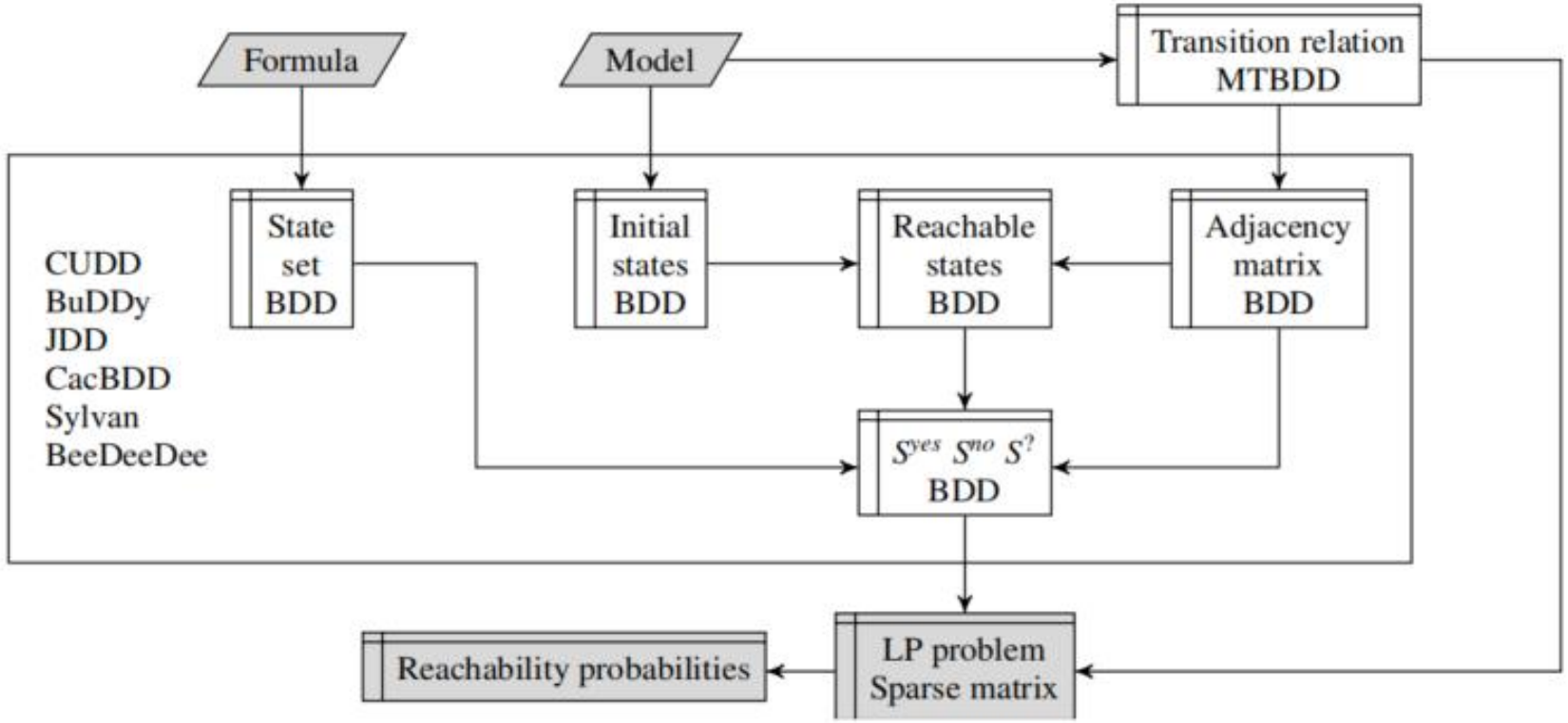
- **Example:**
- x(r1,r2,s1,s2) = 1 if and only if the state encoded by r1r2 has an edge to the state encoded by s1s2



A relation (left) and its BDD representation (right)

PART2 -- FSM model reader

- **Example:** architecture of one of the model checkers



Overview of the data structures used by the model checking algorithm

PART3 -- Formula Evaluation

- **Definition:** Evaluating the CTL formulas against the FSM model to determine their validity.
- **Key Techniques:**

1. Fixed-Point Iteration

2. Recursively traverse the AST

Part1: $True\ False\ Atom\ \neg\ \wedge\ p0 = Img$

Part2: $EX\ EG\ EU\ FU\ FG$

3. Partitioned transition relation

$$\mathbf{EX}f = Pref$$
$$\mathbf{E}[g\mathbf{U}f] = \mathbf{lfp}Z[f \vee (g \wedge \mathbf{EX}Z)]$$
$$\mathbf{EG}f = \mathbf{gfp}Z[f \wedge \mathbf{EX}Z]$$

$$FwdUntil(p,q) = \mathbf{lfp}Z\,[p \vee Img(Z \wedge q)]$$

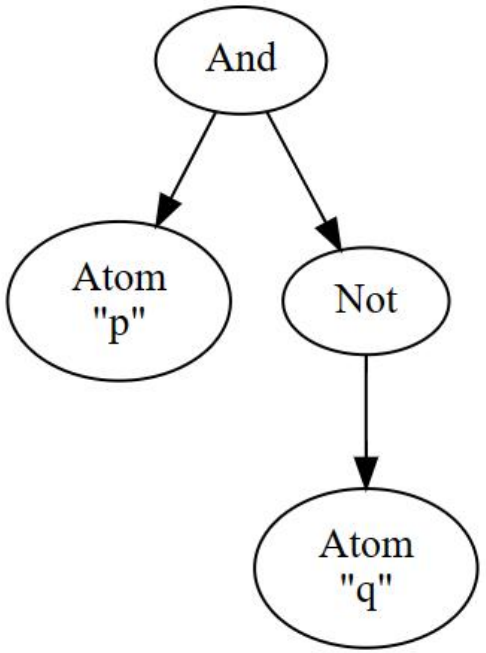
$$EH(p) = \mathbf{gfp}Z[p \wedge Img(Z)]$$
$$Reachable(p,q) = FwdUntil(p;q) \wedge q$$
$$FwdGlobal(p,q) = EH(Reachable(p,q))$$

PART3 -- Formula Evaluation

Key Techniques: Recursively traverse the AST

Part1: $True\ False\ Atom\ \neg\ \wedge\ p0 = Img$

Part2: $EX\ EG\ EU\ FU\ FG$



traverse the AST

PART3 -- Formula Evaluation

- **Key Techniques:** Partitioned transition relation

let: $V = \{v_0, ..., v_{n-1}\}$ $V' = \{v'_0, ..., v'_{n-1}\}$
v'_i's value in the next state: $v'_i = f_i(V)$
define the relation:

$$N_i(V, V') = (V'_i = f_i(V))$$

transition relation for the circuit:

$$N(V, V') = N_0(V, V') \wedge \ldots \wedge N_{n-1}(V, V')$$

Each Ni can usually be represented by a small BDD (less than 100 nodes);
When it bocomes very big, we use partitioned transition relation.

PART4 -- Testing

- **Definition:** Testing and verifying the implementation of the CTL model checker.

- **Key Techniques:**

1. Test Case Generation:
More than 30 tests to evaluate various CTL formulas against several different models

2. Reachability Analysis:
The percentage of test coverage and the coverage report

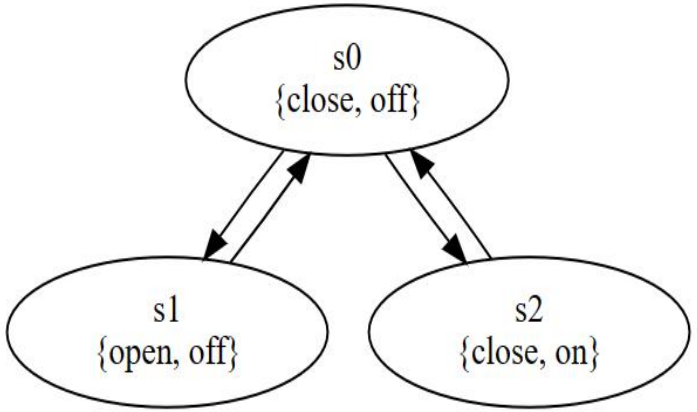
PART4 -- Testing

1. Test Case Generation

- **Example:** Microwave
- Kripke structure K=<S, S0, R, L>

FSM model1:

3
S0 2 colse off
S1 2 open off
S2 2 close on
4
S0 S1
S1 S0
S0 S2
S2 S0



$$s0 \models close \wedge off \iff true$$
$$s0 \models EX(close \wedge on) \iff true$$
$$s0 \models EX(open \wedge off) \iff true$$
$$s0 \models AG(close \vee off) \iff true$$
$$s0 \models EF(open \wedge on) \iff false$$

1. AST construction: AST ast("$AF(p \wedge AXq \Rightarrow EGr)$")
2. FSM model reader: FSM fsm(model1)
3. Formula evaluation: evaluate_formula(ast, fsm)



PART4 -- Testing

2. Reachability Analysis

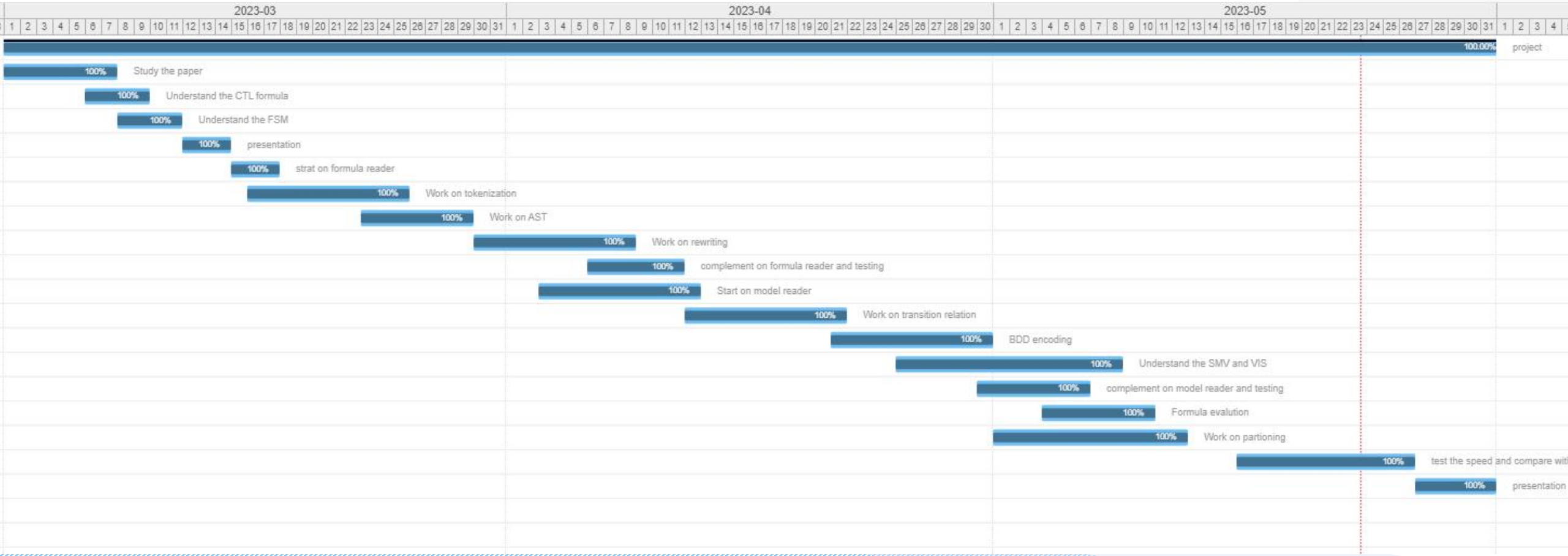
LCOV - code coverage report

Current view:	top level		Hit	Total	Coverage
Test:	coverage.info	**Lines:**	447	508	**88.0 %**
Date:	2023-05-29 16:01:26	**Functions:**	59	65	**90.8 %**

Directory	Line Coverage		Functions	
lib/parsers/cli	**0.0 %**	0 / 34	**0.0 %**	0 / 2
lib/parsers/fsm	**88.5 %**	100 / 113	**88.2 %**	15 / 17
lib/checker	**95.1 %**	58 / 61	**100.0 %**	10 / 10
lib/parsers/ast	**96.3 %**	289 / 300	**94.4 %**	34 / 36

Generated by: LCOV version 1.14

Sceldule


2023-03
2023-04
2023-05
100.00% project
100% Study the paper
100% Understand the CTL formula
100% Understand the FSM
100% presentation
100% strat on formula reader
100% Work on tokenization
100% Work on AST
100% Work on rewriting
100% complement on formula reader and testing
100% Start on model reader
100% Work on transition relation
100% BDD encoding
100% Understand the SMV and VIS
100% complement on model reader and testing
100% Formula evalution
100% Work on partioning
100% test the speed and compare wit
100% presentation

Conclusion

- **Conclusion:**
  1. Implemented techniques for validating CTL properties using forward state traversal

  2. Can be combined with BDD-based state traversal techniques

  3. Become a very practical method for validating large FSMs

- **Future work and Challenge:**
  1. Reading and processing of large-scale FSM models

  2. Parallel and distributed processing

THANK YOU